From IaC Security Smell Detection to Agentic Remediation

State of the Art and Research Methodology

Detecting, analyzing, and reliably remediating Infrastructure-as-Code security misconfigurations

Ahmedou Yahye Kheyri
Supervised by Prof. Hala Bezine
Faculty of Sciences of Sfax — Master MRSI
May 2026

Research Question

State-of-the-art question

What existing methods detect, analyze, and remediate Infrastructure-as-Code security misconfigurations, and what are their limits?

Scientific position

Existing IaC security tools detect misconfigurations, and LLM/RAG systems can suggest fixes, but reliable automatic remediation requires a validation loop with external analyzers.

1. IaC Security Problems

Why IaC matters

IaC scripts such as Terraform, Kubernetes YAML, Ansible, Dockerfiles, and CloudFormation configure real infrastructure directly.

Typical security risks

- hardcoded secrets
- public access and weak network exposure
- excessive privileges
- weak encryption settings
- privileged containers
- missing integrity checks

Core challenge

Misconfigurations are dangerous, frequent, and often context-dependent.

Implication for research

The problem is not only detecting insecure IaC, but also producing trustworthy remediation that works on the real artifact.

2. Security Smells and Taxonomies

Period	Main idea
Early works	Security smells studied in Puppet, Ansible, and Chef scripts.
Seven Sins era	Small taxonomies of recurring IaC security smells became established.
GLITCH era	Polyglot detection across multiple IaC technologies.
Recent updates	Larger smell taxonomies with broad tool coverage, reaching 62 categories across seven IaC tools.

Main observation

Detection taxonomies are becoming mature, but remediation remains much less explored.

3. Static Analysis Tools for IaC

Representative tools

- Checkov
- Terrascan
- tfsec / Trivy
- KICS
- GLITCH
- policy-as-code tools

Limits

- strong at detecting known rule violations
- often limited to alerts or generic advice
- weak contextual remediation support
- little verification of automated fixes

Gap

Static analyzers are effective for detection, but remediation usually remains manual, rule-based, or weakly contextualized.

4. LLMs for Code Security and IaC

What LLMs add

LLMs can generate code, explain vulnerabilities, and propose patches for insecure artifacts.

- useful for patch suggestion and security explanation
- able to leverage natural-language and code context
- recent IaC-focused work such as GenSIaC moves toward security-aware generation
- but generated fixes may hallucinate, fail validation, or introduce new problems

Limit

LLM-based remediation is promising, but textually plausible fixes are not necessarily operationally correct or secure.

5. RAG for Security and Code Assistance

Basic idea

RAG grounds generation by retrieving external knowledge instead of relying only on the model's internal parameters.

- secure IaC examples
- scanner rule documentation
- CWE descriptions
- before/after security fixes
- cloud security guidelines and best practices

Limit

Basic RAG improves grounding, but retrieval can still be noisy, incomplete, or irrelevant, which means the final patch can still fail.

6. Advanced RAG: Self-RAG and CRAG

Why advanced RAG matters

More recent systems do not trust one retrieval pass and one generation pass.

- Self-RAG adds self-reflection and critique
- CRAG adds retrieval evaluation and corrective retrieval behavior
- both move from passive retrieval to controlled, quality-aware generation

Relevance to this work

These ideas motivate a remediation loop in which retrieved knowledge, generation quality, and correction are treated as uncertain.

7. Tool-Using and Agentic LLMs

Tool use

Toolformer and later agentic systems show that language models can call external tools instead of acting as pure text generators.

Relevant tools for IaC

- Checkov
- KICS
- Terrascan
- Terraform validators
- YAML and manifest checkers
- policy engines

Key idea

The language model can act as an agent that interacts with analyzers to verify and improve candidate fixes.

State-of-the-Art Logic

IaC is widely used, but misconfigurations are dangerous.

↓

Research identifies IaC security smells and taxonomies.

↓

Static analyzers detect many problems, but remediation remains limited.

↓

LLMs can suggest fixes, but they may hallucinate or break the artifact.

↓

RAG improves grounding, but retrieval itself can fail.

↓

Advanced RAG and agents add reflection, correction, and tool use.

↓

Therefore: reliable IaC remediation should combine generation with external validation and iteration.

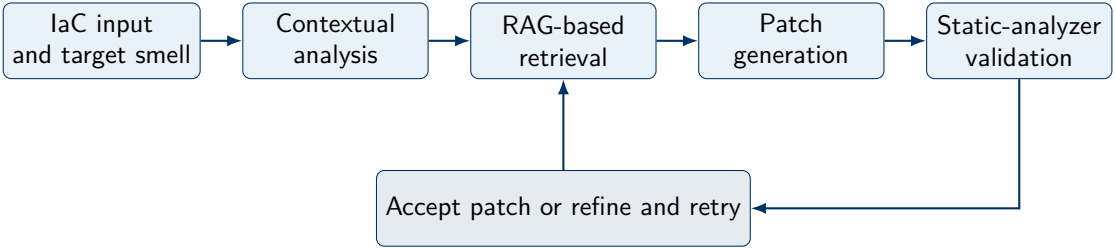
Main Research Gap

Gap statement

Existing IaC security research has made strong progress in smell detection, taxonomy construction, and static analysis. However, less attention has been given to reliable automatic remediation.

- static analyzers identify misconfigurations but usually provide limited contextual fixes
- LLM-based approaches can generate patches but may hallucinate or introduce new errors
- reliable remediation therefore requires retrieval, generation, validation, and iterative improvement

Proposed Framework



- candidate patches are accepted only after external validation
- the remediation loop combines retrieval, generation, testing, and refinement

Evaluation Logic

Metric family	Question answered
Patch validity	Is the produced patch syntactically and operationally acceptable?
Smell elimination	Does the targeted issue disappear after remediation?
No-new-issues	Does the fix avoid introducing additional scanner findings?
Cross-validator agreement	Do Checkov and KICS support the same acceptance decision?
Qualitative failure analysis	Why do failures happen: retrieval mismatch, weak patch, validator conflict, or incomplete context?

Experimental Direction

Evaluation objective

The framework must be assessed empirically on real IaC artifacts and validated with independent analyzers.

- freeze the training and evaluation splits
- prepare Kaggle experiments for QLoRA fine-tuning
- evaluate selected system configurations
- collect quantitative results and qualitative remediation examples
- compare patch validity, smell elimination, and no-new-issues behavior

Expected Outcomes

Scientific contribution

- focused state of the art
- explicit research gap
- methodology for validated remediation
- clear positioning in the literature

Empirical contribution

- Kaggle training logs
- validated test results
- comparison tables
- journal-ready figures and discussion

Bottom line

The research positioning is clear, but the strength of the final contribution depends on rigorous experimental validation.

Thank you

Questions and discussion